Comp683: Computational Biology

Lecture 9

February 18, 2026

Today + announcements

- **Announcement**: Homework 1 is available on github. Please turn in by a PDF by 11:59pm on Wed March 4 on canvas.
- **Announcement**: Journal club presentation on Monday!
- Finish single-cell data augmentation
- Start graph signal processing (GSP) overview
- Give some background required for our journal club paper monday

Review on the sparse data landscape problem

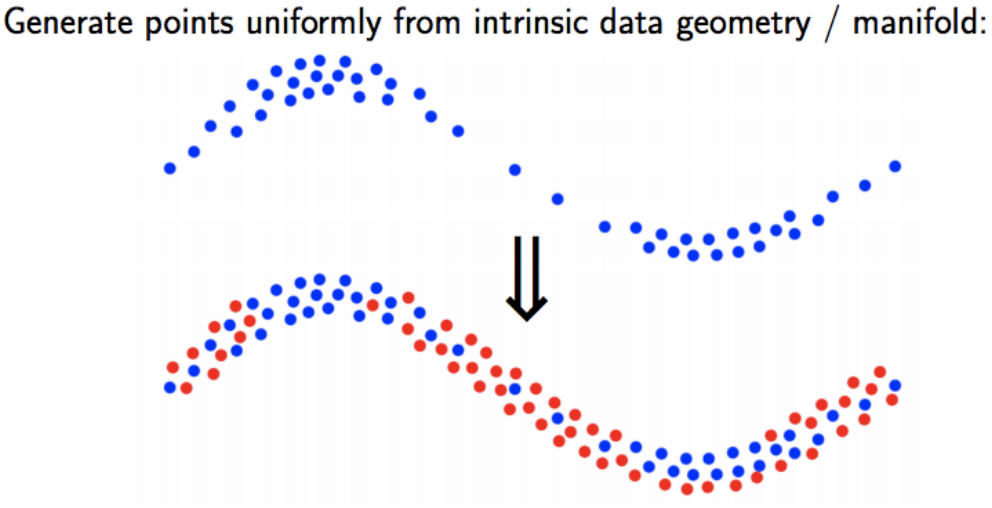


Figure: from Lindenbaum *et al.* NeurIPS. 2018

Synthesis Using Geometrically Aligned Random Walks

Algorithm 1 SUGAR: Synthesis Using Geometrically Aligned Random-walks

Input: Dataset $\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}, \boldsymbol{x}_i \in \mathbb{R}^D$.

Output: Generated set of points $\boldsymbol{Y} = \{\boldsymbol{y}_1, \boldsymbol{y}_2, \ldots, \boldsymbol{y}_M\}, \boldsymbol{y}_i \in \mathbb{R}^D$.

1: Compute the diffusion geometry operators $\boldsymbol{K}$, $\boldsymbol{P}$, and degrees $\hat{d}(i)$, $i = 1, ..., N$ (see Sec. 3)

2: Define a sparsity measure $\hat{s}(i), i = 1, ..., N$ (Eq. 2).

3: Estimate a local covariance $\boldsymbol{\Sigma}_i$, $i = 1, ..., N$, using k nearest neighbors around each $\boldsymbol{x}_i$.

4: For each point $i = 1, ..., N$ draw $\hat{\ell}(i)$ vectors (see Sec. 4.3) from a Gaussian distribution $\mathcal{N}(\boldsymbol{x}_i, \boldsymbol{\Sigma}_i)$. Let $\hat{\boldsymbol{Y}}_0$ be a matrix with these $M = \sum_{i=1}^{N} \hat{\ell}(i)$ generated vectors as its rows.

Figure: from Lindenbaum *et al.* NeurIPS. 2018. Let's walk through steps 1-4 for now.

Unpacking details

- Let $\mathbf{Y}_0$ be the set of all new $M = \sum_i \ell(i)$ points generated around each i, with $\mathbf{Y}_0 = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_M\}$
- **MGC Kernel:** Now use affinities between points in $\mathbf{X}$ and $\mathbf{Y}_0$. Here, points in $\mathbf{X}$ are used as reference.

$$\hat{\mathcal{K}}(\mathbf{y}_i, \mathbf{y}_j) = \sum_r \mathcal{K}(\mathbf{y}_i, \mathbf{x}_r)\mathcal{K}(\mathbf{x}_r, \mathbf{y}_j)s(r) \tag{1}$$

Pulling $\mathbf{Y}_0$ towards sparser regions of $\mathcal{M}$

Pasted psuedo code for the second part of SUGAR

5: Compute the sparsity based diffusion operator $\hat{\boldsymbol{P}}$ (see Sec 4.2).
6: Apply the operator $\hat{\boldsymbol{P}}$ at time instant t to the new generated points in $\hat{\boldsymbol{Y}}_0$ to get diffused points as rows of $\boldsymbol{Y}_t = \hat{\boldsymbol{P}}^t \cdot \boldsymbol{Y}_0$.
7: Rescale $\boldsymbol{Y}_t$ to get the output $\boldsymbol{Y}[\cdot, j] = \boldsymbol{Y}_t[\cdot, j] \cdot \frac{\text{percentile}(\boldsymbol{X}_{[\cdot,j]},.99)}{\max \boldsymbol{Y}_{t[\cdot,j]}}$, $j = 1, \ldots, D$, in order to fit the original range of feature values in the data.

Figure: from Lindenbaum *et al.* NeurIPS. 2018.

Diffusion Operator Again

- Take affinities from $\hat{\mathcal{K}}$ and convert them to $\mathbf{P}$, the row-normalized Markov matrix.
- This will allow us to correct points in $\mathbf{Y}_0$ according to neighborhood regions

As you will all recognize, powering $\mathbf{P}$ as $\mathbf{P}^t$ estimates the probability of successfully traveling between nodes with t steps. The transformed matrix, $\mathbf{Y}_t$ is computed as

$$\mathbf{Y}_t = \mathbf{P}^t \times \mathbf{Y}_o \tag{2}$$

Same Story Regarding t

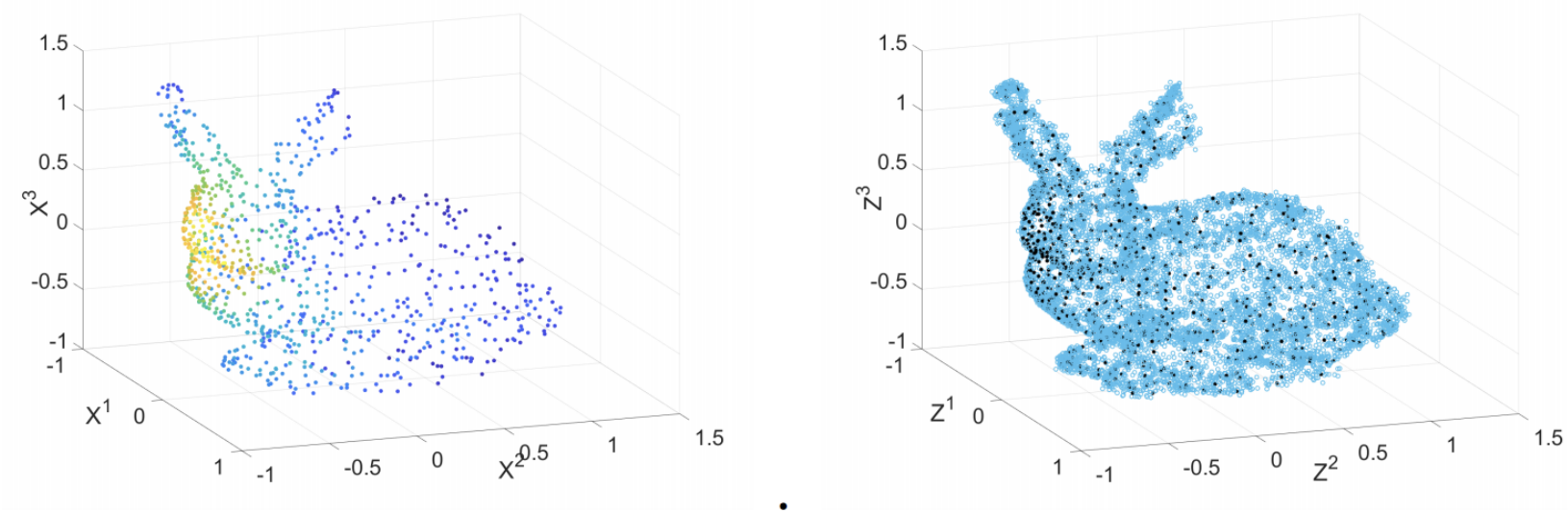
Remember in PHATE, t was chosen according to the knee point of the Von-Neumann entropy of the normalized eigenvalues of $\mathbf{P}^t$.

- Let $[\eta(t)]_i = \lambda_i^t / \sum_{j=0}^{N-1} \lambda_j^t$ be the probability distribution defined by the non-negative eigenvalues of $\mathbf{P}^t$.

$$H(t) = -\sum_{i=1}^{N} [\eta(t)]_i \log([\eta(t)]_i) \tag{3}$$

Experiments : Synthetic and Biological

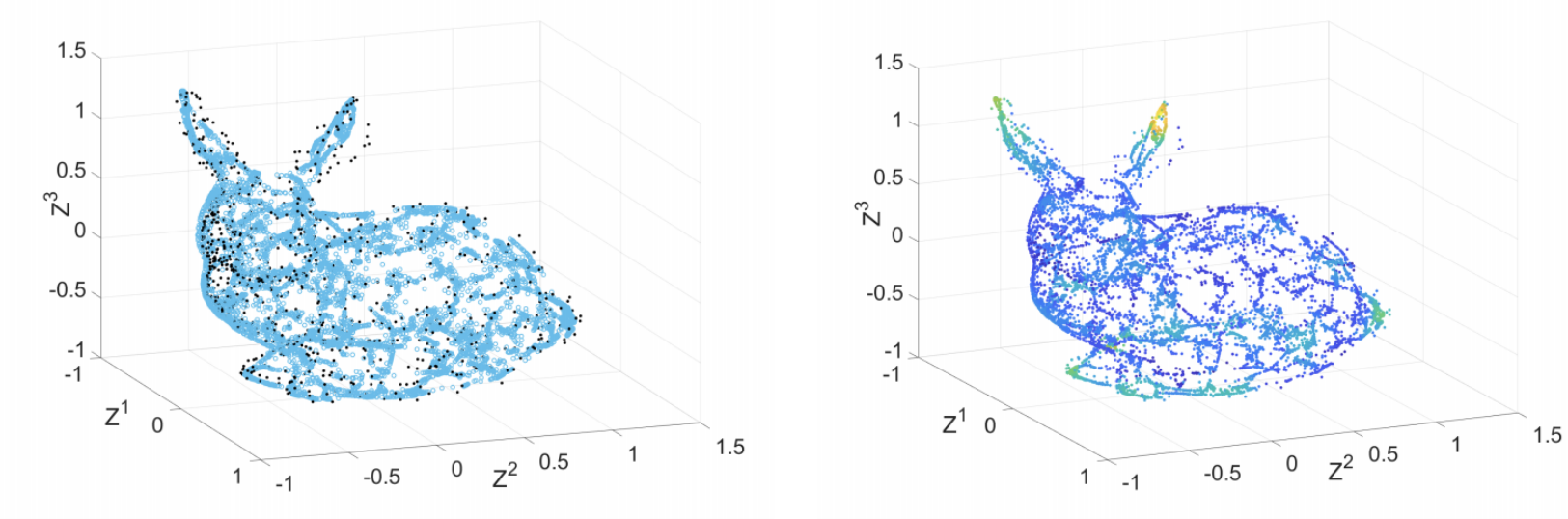
Stanford Bunny



(a) (b)

Figure: from Lindenbaum *et al.* NeurIPS. 2018. (**a**). The original set, $\mathbf{X}$ of points, colored by node degree. (**b**) $\mathbf{Y}_0$ are generated points (blue) and original points, $\mathbf{X}$ (black).

Stanford Bunny Part II



(c) (d)

Figure: from Lindenbaum *et al.* NeurIPS. 2018. (**c**). Original and generated points, before MGC diffusion. (**d**) Generated points (**Y**) after MGC diffusion. Points are colored by degree.

Application : Augmented Clustering

SUGAR was used to generate additional data points to improve the quality of clusters identified with k-means.

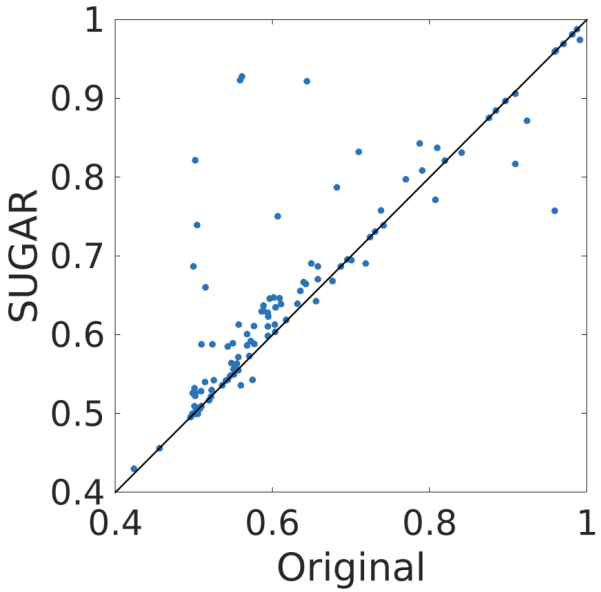
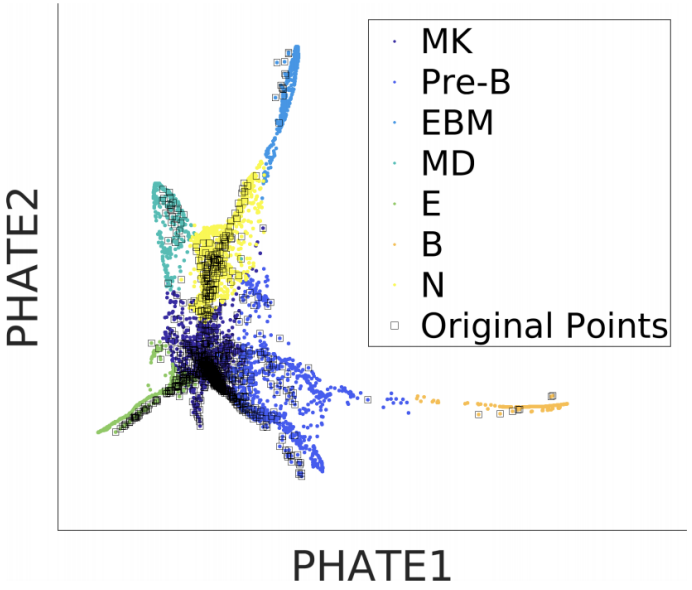


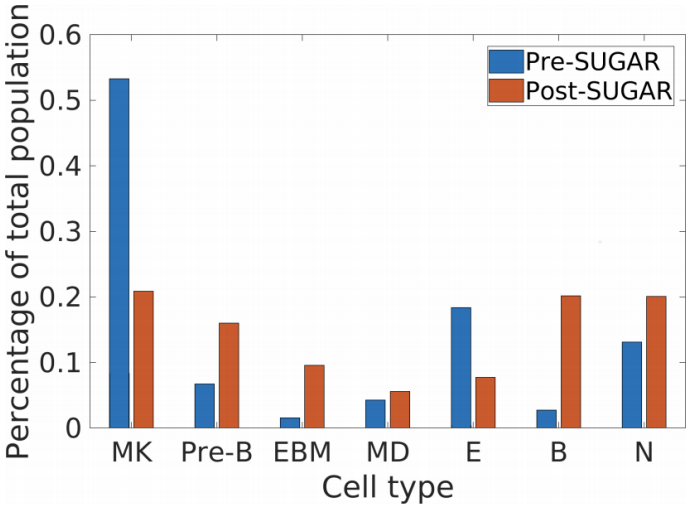
Figure: from Lindenbaum *et al.* NeurIPS. 2018. In 119 datasets, the data were augmented with additional datapoints using sugar. Adjusted Rand Index was computed for the original data vs data + SUGAR.

SUGAR on Single-Cell

SUGAR was using to augment cells in a single-cell RNAseq dataset.



(a)



(b)

Figure: from Lindenbaum *et al.* NeurIPS. 2018

Maintaining Intra-Module Marker Co-Expression

Among cells assigned to the same module or cluster, after SUGAR led to higher intra-module between-marker correlation.

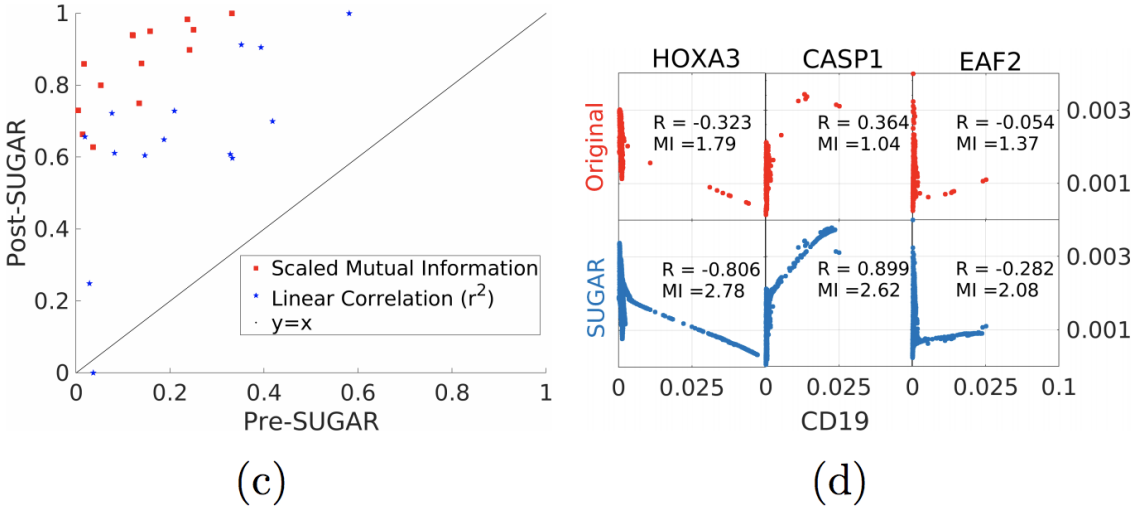


Figure: from Lindenbaum *et al.* NeurIPS. 2018

Graph Signal Processing

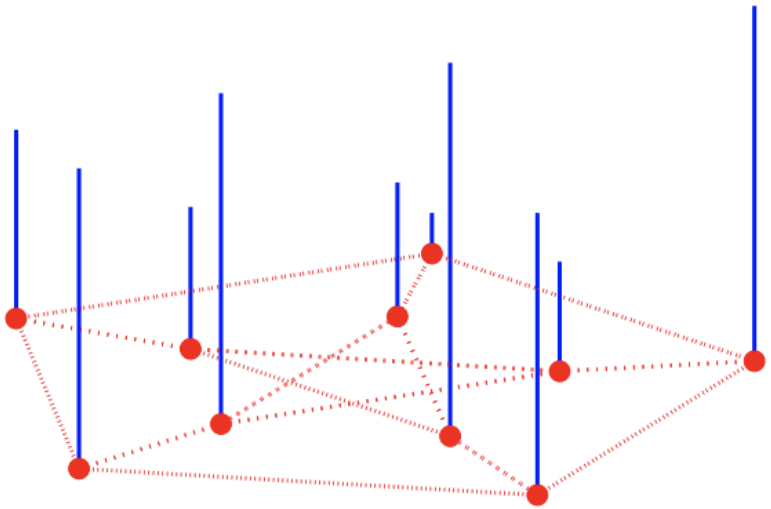


Figure: from Shuman *et al.* ArXiv. The purpose is to study the interplay between some signal (e.g. a clinical outcome or gene expression) and graph connectivity.

The Interplay Between Signal and Graph Structure

Remember, our friend Graph Laplacian ($\mathbf{L} = \mathbf{D} - \mathbf{A}$),

- We will use spectra of the graph Laplacian to link signal, $\mathbf{f}$ to graph structure and modulate as necessary. For example $\mathbf{f}$ could be a clinical label of a cell.
  - First re-write $\mathbf{f}$ in terms of eigenvectors of the Laplacian
  - The eigenvectors corresponding to the first *smallest* eigenvalues of $\mathbf{L}$ are considered **low frequency**, and hence entries of the eigenvector entries corresponding to nodes that are connected should be similar
  - For higher **high frequencies** corresponding to larger eigenvalues, the values of the eigenvectors of adjacent nodes will be more different.

Signal Specificity

Here we visualize eigenvector entries at nodes ($\mathbf{u}_0$,$\mathbf{u}_1$,$\mathbf{u}_{50}$)

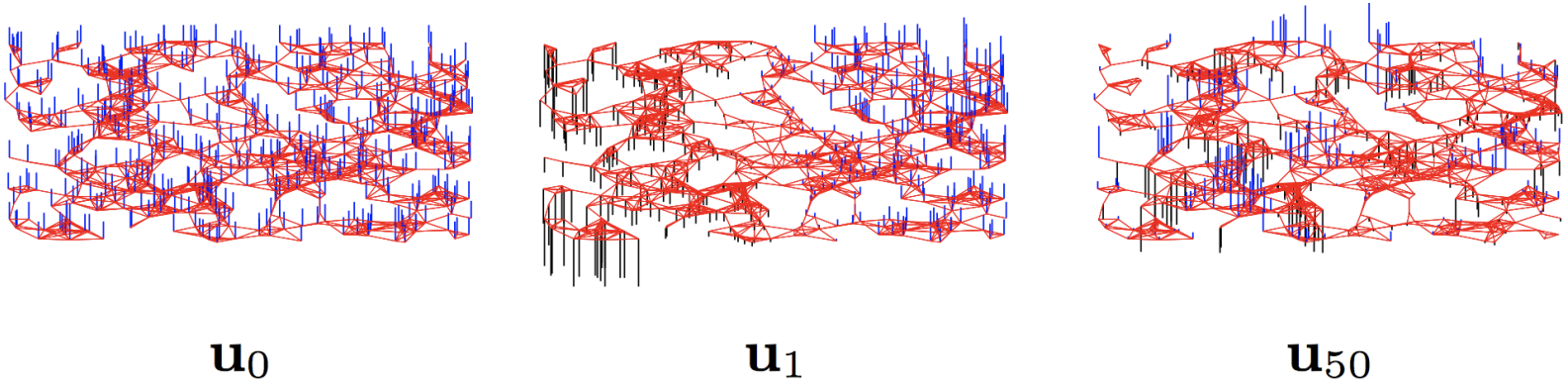


Figure: from GSP Review https://arxiv.org/abs/1211.0053

Same Concept Visualized A Bit Differently

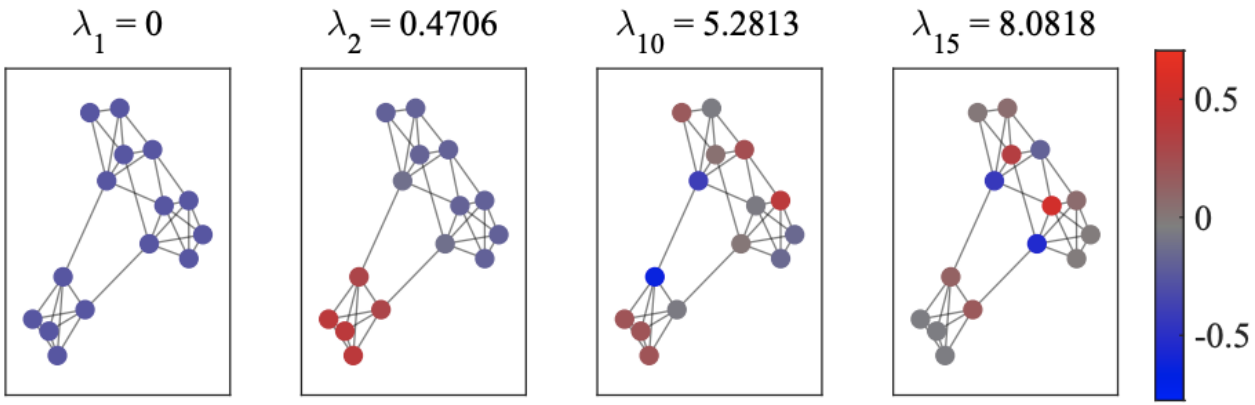


Figure: Notes are colored by their corresponding eigenvector component. From `https://arxiv.org/pdf/2008.01305.pdf`

Similarly

Zero crossings mean that eigenvector entries are neighboring nodes will be different.

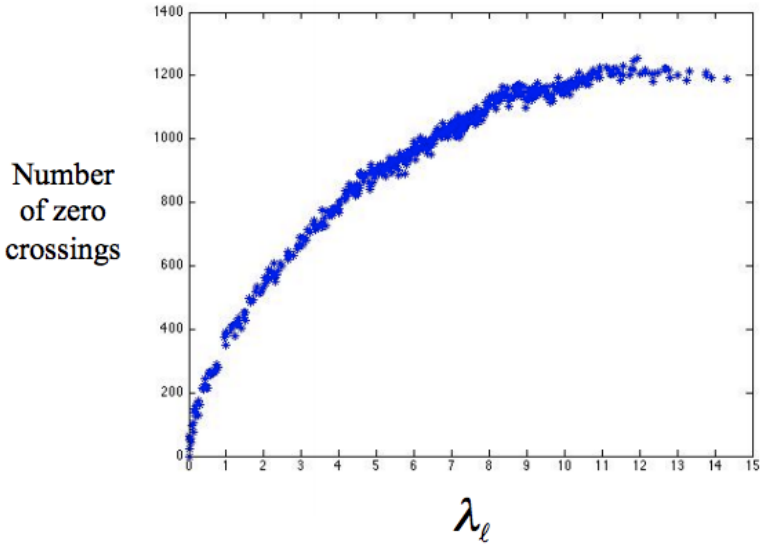


Figure: from GSP Review https://arxiv.org/abs/1211.0053

What is Graph Fourier Transform (on a high level?)

- Explain frequency content of the graph signal (e.g. experimental measurements/labels/etc) as a weighted sum of the eigenvectors of the Graph Laplacian
- The eigenvectors of the Graph Laplacian comprise the **Graph Fourier Basis and can help to decouple high and low frequency signals**

Local Variation of a Signal

The local variation of a signal or the sum of differences around a node can be written as,

$$
\begin{aligned}
(\mathcal{L}\mathbf{f})(i) &= ([\mathbf{D} - \mathbf{A}]\mathbf{f})(i) && (4)\\
&= d(i)\mathbf{f}(i) - \sum_j A_{ij}\mathbf{f}(j) && (5)\\
&= \sum_j A_{ij}(\mathbf{f}(i) - \mathbf{f}(j)) && (6)
\end{aligned}
$$

Local Variation Leads to Total Variation

The total variation of a signal on a graph is defined as follows and is also known as the Laplacian Quadratic Form

$$TV(\mathbf{f}) = \sum_{i,j} A_{ij}(\mathbf{f}(i) - \mathbf{f}(j))^2 \tag{7}$$

$$= \mathbf{f}^T \mathcal{L} \mathbf{f} \tag{8}$$

- Note here I have been assuming that we have an unweighted graph, but you could certainly substitute A_{ij} with a weighted version, W_{ij}